# CoinStream Architecture & Implementation Details

## 1. Project Overview

CoinStream is a Next.js 16 application that acts as a real-time crypto analytics dashboard. It utilizes the CoinGecko API for both REST-based historical data and WebSocket-based live streams. The UI is built with Tailwind CSS v4, shadcn/ui, and Lucide icons.

## 2. File Structure & Routing

The app follows the Next.js App Router paradigm:

- `app/layout.tsx`: Root layout, configures Geist fonts and global CSS. Includes the `Header` component.
- `app/page.tsx`: Home page featuring `CoinOverview` (Bitcoin stats), `TrendingCoins`, and `Categories`.
- `app/coins/page.tsx`: A paginated table of the top cryptocurrencies using `/coins/markets` endpoint.
- `app/coins/[id]/page.tsx`: Dynamic coin detail page displaying live data, trades, candlestick charts, and a currency converter.

## 3. Data Fetching (REST API)

All server-side REST API calls are centralized in `lib/coingecko.actions.ts`.

- It uses a custom `fetcher` function that wraps the native `fetch` API.
- Query parameters are stringified using the `query-string` package.
- It injects the `x-cg-demo-api-key` (or `x-cg-pro-api-key` for Pro users) header into every request.
- Handles endpoints like `/coins/markets`, `/search/trending`, `/coins/categories`, and `/coins/{id}/ohlc`.

## 4. WebSocket Implementation (`hooks/useCoinGeckoWebSocket.ts`)

The real-time data flow is managed by a custom React hook `useCoinGeckoWebSocket`.

- **Connection**: It establishes a connection to `wss://ws.coingecko.com/cable` with the API key passed as a URL parameter.
- **Subscription Management**:
  - Maintains a `subscribed` Set (via `useRef`) to track active channels.
  - Subscribes to `CGSimplePrice` for live price updates.
  - Subscribes to `OnchainTrade` and `OnchainOHLCV` (if a `poolId` is available) for live DEX trades and live candlestick updates.
- **Message Handling**:
  - `ping/pong`: Automatically replies to `ping` messages to keep the connection alive.
  - `C1` messages: Parses live price updates (price, 24h change, market cap, volume).
  - `G2` messages: Parses live trade executions (buy/sell, price, amount, timestamp) and maintains a rolling list of the 7 most recent trades.
  - `G3` messages: Parses live OHLCV (Open, High, Low, Close, Volume) data for the current candle.
- **Cleanup**: Unsubscribes from all channels and closes the WebSocket connection when the component unmounts or dependencies change.

## 5. Candlestick Chart Implementation (`components/CandlestickChart.tsx`)

The application uses TradingView's `lightweight-charts` library to render the candlestick charts.

- **Initialization**:
  - A chart instance is created inside a `useEffect` and attached to a `div` container via a ref.
  - The chart's visual configuration (colors, grid, crosshair) is defined in `constants.ts`.
- **Data Merging**:
  - The chart accepts historical OHLC data fetched via the REST API.
  - It also accepts `liveOhlcv` from the WebSocket.
  - In a `useEffect`, it merges the historical data with the live candle. If the live candle's timestamp matches the last historical candle, it replaces it; otherwise, it appends it.
- **Timeframe Selection**: Users can switch between periods (1D, 1W, 1M, etc.). This triggers a new REST fetch for the corresponding historical OHLC data, which is then updated on the chart via `startTransition`.
- **Responsive Design**: A `ResizeObserver` ensures the chart automatically resizes to fit its container when the window or layout changes.

## 6. Real-Time UI (`components/LiveDataWrapper.tsx`)

This component acts as the orchestrator for a specific coin's live view.

- It calls `useCoinGeckoWebSocket` to get `price`, `trades`, and `ohlcv`.
- It passes the live price to the `CoinHeader`.
- It passes the `ohlcv` to the `CandlestickChart`.
- It renders a `DataTable` showing the live stream of recent buy/sell trades dynamically updating as WebSocket messages arrive.

## 7. Search Modal (`components/SearchModal.tsx`)

- A custom modal that opens via the navigation bar or the `Cmd+K` / `Ctrl+K` shortcut.
- Uses a debounced (300ms) call to the `/search` endpoint to find coins by name or symbol.
- Supports keyboard navigation (Arrow Up/Down, Enter) to select a coin and route to its detail page.

## 8. Styling & Theming

- Built primarily with Tailwind CSS v4 using CSS variables for colors (defined in `app/globals.css`).
- Utilizes `clsx` and `tailwind-merge` (via the `cn` utility) for dynamic class string construction.
- Components from `shadcn/ui` (Table, Input, Select, Pagination, Badge) are customized to match the dark, neon-accented theme of the application.